

DOKUMENTATION

Aufgabe 3 – Evaluation verschiedener Trainings-Settings

3a) Getestete Settings und erreichte Genauigkeiten

Zur Untersuchung des Einflusses verschiedener Hyperparameter auf die Modellleistung wurden vier unterschiedliche Trainings-Settings getestet.

Dabei wurden Optimierer, Lernrate und L2-Regularisierung variiert.

Alle Modelle besitzen dieselbe Architektur (MLP mit zwei Hidden-Layern mit 16 Neuronen) und wurden mit identischen Trainings-, Validierungs- und Testdaten trainiert.

Netzarchitektur (für alle Settings identisch):

- Input: 30 Features
- Hidden Layer 1: Dense(16), ReLU
- Hidden Layer 2: Dense(16), ReLU
- Output Layer: Dense(2), Softmax
- Loss-Funktion: categorical_crossentropy
- Early Stopping (patience = 80, monitor = val_loss)

Übersicht der getesteten Settings:

Setting	Optimierer	Optimierer Learnrate	L2- Regularisierung	Test-Accuracy
1	Adam	0.001	0.0	100%
2	Adam	0.0005	0.0001	98.83%
3	SGD	0.01	0.0001	98.83%
4	Adam	0.001	0.0001	98.83%

3b) Erreichen der geforderten Genauigkeit

Die geforderte Mindestgenauigkeit von 98 % wurde in allen Settings erfolgreich erreicht.

Will man zum Beispiel die Genauigkeit des ersten Settings auf Testdaten überprüfen, so könnte man schon die Datei „do_prediction.py“ ausführen lassen. Für die anderen, soll man in derselben Datei den Pfad zum ersten Modellgewichte durch anderen Pfad in der Zeile „myModel.load_weights("models/setting_1_best.weights.h5")“ ersetzen. Die entsprechenden Modellgewichte findet man in dem Verzeichnis „models“...

Damit erfüllt das entwickelte Modell die Anforderungen der Aufgabenstellung.

3c) Erklärung der beobachteten Effekte und Code-Dokumentation

Einfluss des Optimierers

- **Adam** zeigt in allen getesteten Konfigurationen ein schnelles und stabiles Konvergenzverhalten.
Besonders in Setting 1 erreicht das Modell bereits nach wenigen Epochen eine gute Testgenauigkeit.
- **SGD (Setting 3)** benötigt mehr Epochen zur Konvergenz, erreicht jedoch ebenfalls eine gute Endgenauigkeit.
Dies ist typisch, da SGD empfindlicher gegenüber Lernrate und Initialisierung ist, dafür aber oft gute Generalisierungseigenschaften besitzt.

Einfluss der Lernrate

- Eine zu hohe Lernrate kann zu instabilem Training führen, während eine zu niedrige Lernrate die Konvergenz verlangsamt.
- Die getesteten Lernraten (**0.001, 0.0005 und 0.01 bei SGD**) stellen einen guten Kompromiss dar:
 - **0.001 (Adam)** erwies sich als besonders effektiv
 - **0.0005** führte zu stabilerem, aber langsamerem Lernen

Einfluss der L2-Regularisierung

- Die L2-Regularisierung reduziert **Overfitting**, indem große Gewichte bestraft werden.
- In den Lernverläufen ist zu erkennen:
 - Ohne **L2 (Setting 1)** steigt die Trainingsgenauigkeit sehr schnell auf **100 %**, jedoch bleibt die Validierungsleistung stabil -- > kein kritisches **Overfitting**
 - Mit **L2 (Settings 2–4)** steigt die Loss-Funktion wieder langsamer nachdem das minimum erreicht wurde, die Generalisierung bleibt jedoch nicht schlecht.

Beobachtung des Early Stopping

In allen Settings wurde **Early Stopping** erfolgreich eingesetzt.

Nach Erreichen eines **Minimums** der Validierungs-Loss stieg diese wieder an, während die **Trainingsgenauigkeit** weiter zunahm – ein klares Zeichen für beginnendes **Overfitting**.

Durch das automatische Wiederherstellen der besten **Gewichte** wurde stets das **optimal generalisierende Modell** verwendet.

Bezug zum Code :

a) Preprocessing :

Zunächst werden die Eingabedaten standardisiert, indem für jedes Feature Mittelwert und Standardabweichung über den Trainingsdaten berechnet und anschließend alle Datensplits entsprechend skaliert werden. Dadurch liegen alle Features ungefähr im

Bereich mit Mittelwert 0 und Varianz 1, was den Lernprozess des neuronalen Netzes mit Adam deutlich stabilisiert.

b) **Modell (annClassifier) :**

Das neuronale Netz besteht aus einem Input-Layer mit 30 Eingangsneuronen, zwei verdeckten Dense-Schichten, je mit ReLU-Aktivierung und 16 Neuronen sowie einem Output-Layer mit zwei Neuronen und Softmax-Aktivierung für die binäre Klassifikation mit One-Hot-Labels. Optional wird eine L2-Regularisierung auf die Gewichtsmatrizen angewendet, um Overfitting zu reduzieren. Optimiert wird mit Adam und der Loss ist categorical_crossentropy.

c) **Training (train_model) :**

Das Training wird durch EarlyStopping und ModelCheckpoint kontrolliert. EarlyStopping überwacht die Validierungsloss, und ModelCheckpoint speichert die jeweils besten Gewichte in eine Datei. Auf diese Weise verwende ich am Ende das Modell mit der höchsten Validierungsleistung und vermeide, dass spätere Epochen mit Overfitting die Ergebnisse verschlechtern

Zusammengefasst:

- **Die L2-Regularisierung** wird in den Dense-Layern über kernel_regularizer=l2(...) realisiert.
- **Lernrate und Optimierer** werden dynamisch über die settings-Liste gesetzt.
- **Early Stopping und ModelCheckpoint** sichern stabile Trainingsverläufe und reproduzierbare Ergebnisse.
- **Die manuelle Standardisierung der Daten** verhindert Daten-Leakage, da ausschließlich Trainingsdaten zur Berechnung von Mittelwert und Standardabweichung verwendet werden.

Ergänzung: Gradio-Oberfläche

Ein Screenshot der entwickelten Gradio-Oberfläche ist unten beigefügt.

Die Oberfläche erlaubt:

- Auswahl von Optimierer, Lernrate und L2-Regularisierung
- Starten des Trainings
- Anzeige der erreichten Genauigkeiten
- Vorhersagen für einzelne Beispiele

Breast Cancer MLP – Hyperparameter-Interface

Training Vorhersage

Modell trainieren

L2-Regularisierungswert

0,0008



Lernrate

0,00751



Optimizer

SGD

Modell trainieren

Trainingsergebnis

Training abgeschlossen.

Optimizer: SGD, Lernrate: 0.00751, L2: 0.0008

Validierungs-Accuracy: 0.9412

Test-Accuracy: 0.9884

Breast Cancer MLP – Hyperparameter-Interface

Training Vorhersage

Vorhersage auf einem Validierungsbeispiel

Index des Validierungsbeispiels

34



Model auswählen

setting_gradio_best.weights.h5

Beispiel vorhersagen

Ergebnis des Vorhersagens

Die Vorhersage

Model : setting_gradio_best.weights.h5

Index in Validierungsmenge: 34

Wahre Klasse: [0. 1.]

Vorhergesagte Klasse: 1

Wahrscheinlichkeiten: $P(0) = 0.001$, $P(1) = 0.999$